

Object-Oriented Programming in Java

Emily Lucia Antosch

17.09.2026

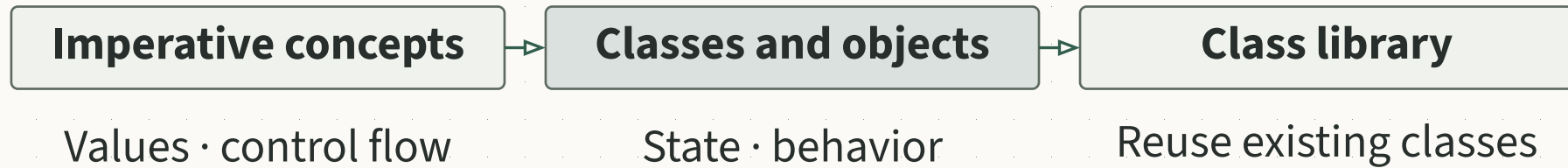
Today's route

1. Introduction	2
2. Classes and Objects	5
3. Variables and Memory	17
4. Methods	31
5. Constructors	59
6. Class Variables & Class Methods	75
7. License Notice	85

1. Introduction

Where Are We Now?

Today



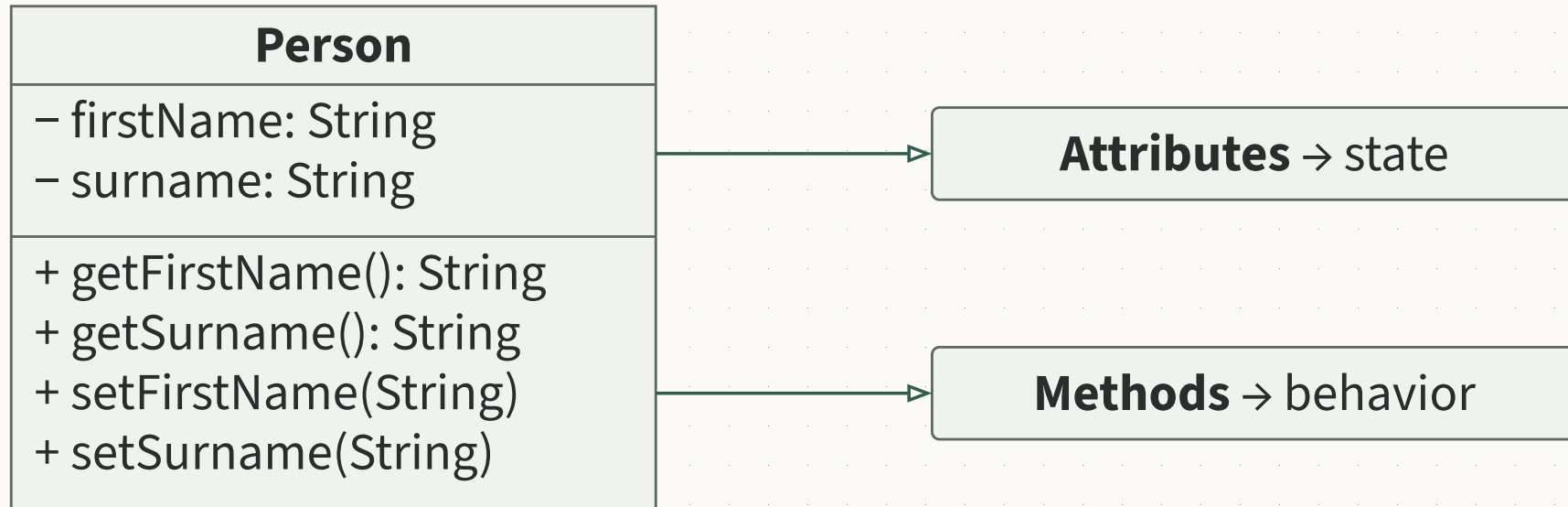
The Goal of This Chapter

- Model data and behavior with classes.
- Create objects and change their state through methods.
- Distinguish instance members from class members.

2. Classes and Objects

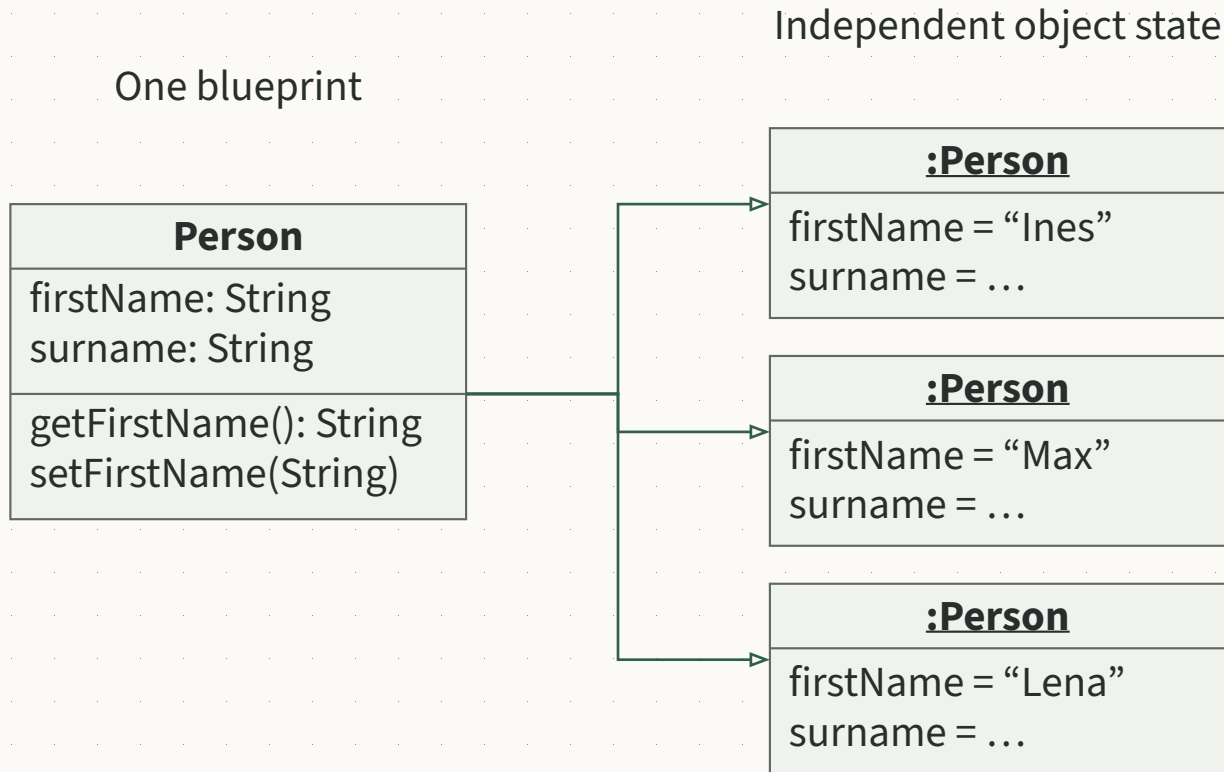
Class

A class is a “blueprint” for objects.



Attributes + methods = members

Relationship Between Class and Object



Classes in Java

```
1 class ClassName {  
2     // Fields  
3     // Methods  
4 }
```

Java



Info

Use one source file per class: `ClassName.java`.

Example: Simple Class



Task 1

Model a student with a name, student number, and enrollment year.

Example: Simple Class

```
1 class Student {  
2     String name;  
3     int matrNumber;  
4     int enrolledYear;  
5 }
```

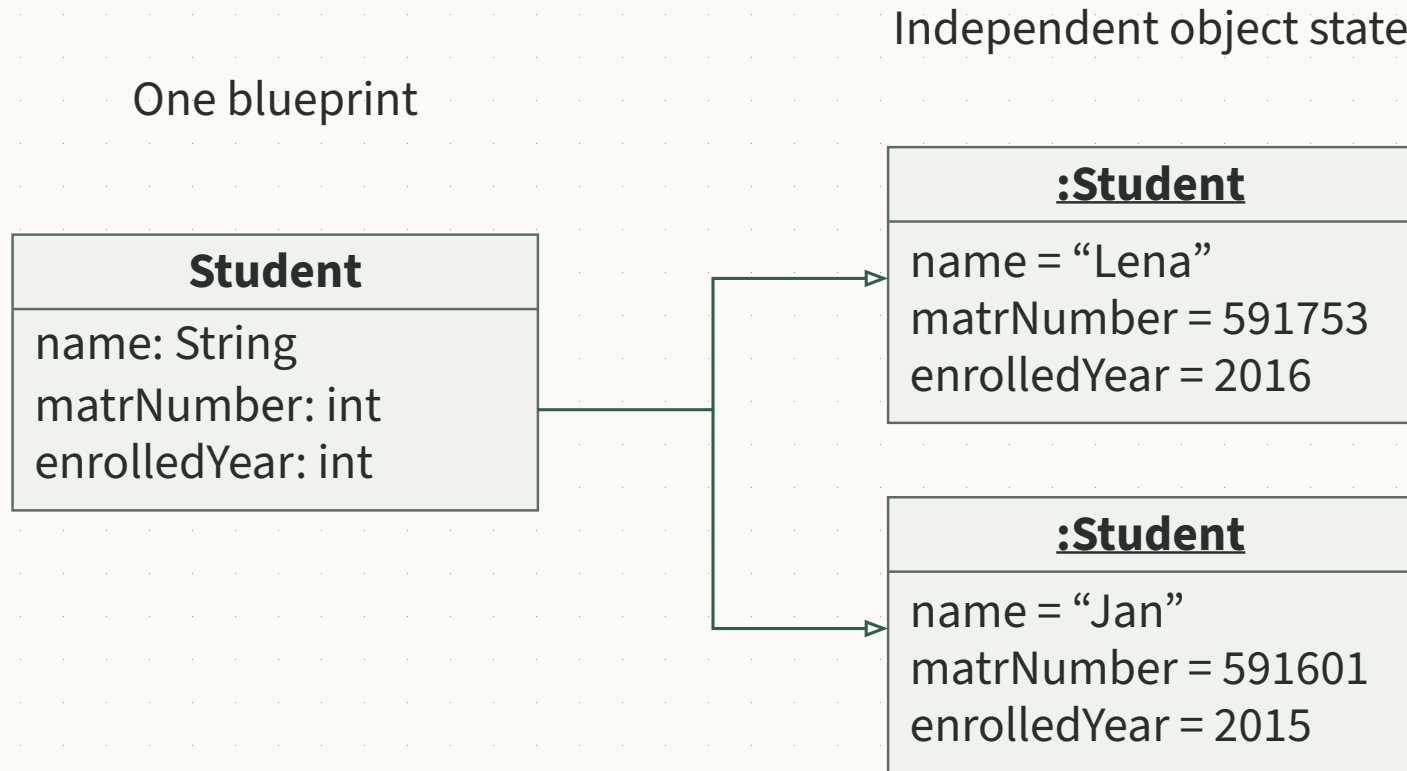
Java

Student

name: String
matrNumber: int
enrolledYear: int

No methods or encapsulation yet.

Example: One Class, Many Objects



Example: Local Variables



What values do `count`, `lena`, and `jan` have?

```
1 public class StudentDemo {  
2     public static void main(String[] args) {  
3         int count;  
4         Student lena, jan;  
5     }  
6 }
```

Java

Example: Local Variables

Local variables

count: int

unassigned

lena: Student

unassigned

jan: Student

unassigned

Object memory

No Student objects yet

Reading an unassigned local variable is a compile error.

Example: new Operator

```
1 public class StudentDemo {  
2     public static void main(String[] args) {  
3         int count;  
4         Student lena, jan;  
5         lena = new Student();  
6     }  
7 }
```

Java

`new Student()` creates an object. = stores its reference.

Example: `new` Operator

1. Create the object: allocate memory and initialize fields.

Local variables

count: int

unassigned

lena: Student

unassigned

jan: Student

unassigned

Object memory

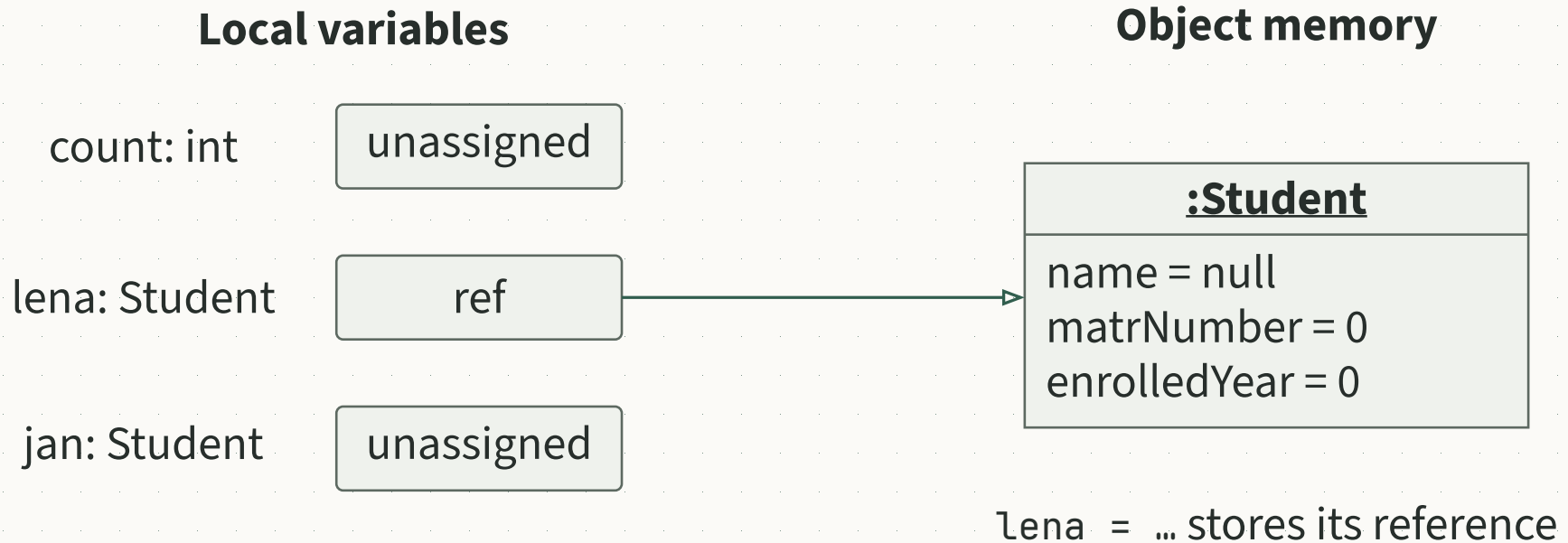
:Student

name = null
matrNumber = 0
enrolledYear = 0

`new Student()` creates an object

Example: `new` Operator

2. Assign the reference to `lena`.

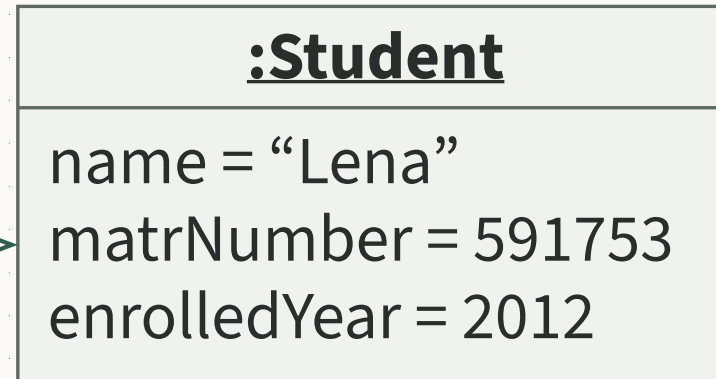
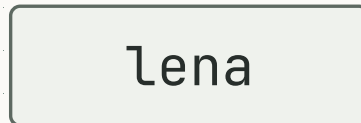


3. Variables and Memory

Access to Object Variables

Use the dot operator: `objectReference.member`.

Object reference



`lena.enrolledYear` → 2012

Access to Object Variables



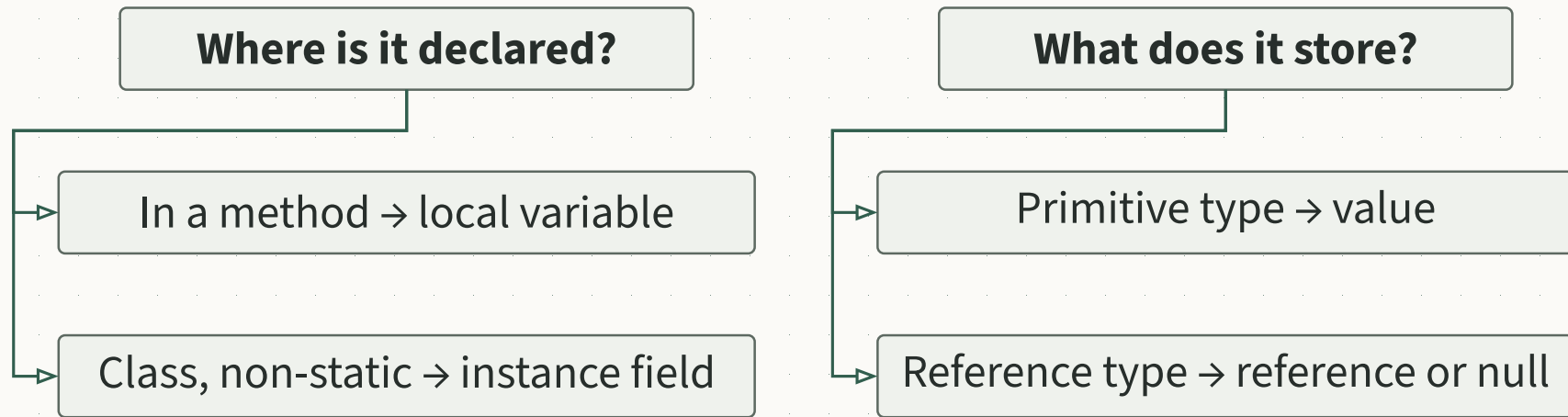
What will this print?

```
1  public class StudentDemo1 {  
2      public static void main(String[] args) {  
3          Student lena = new Student();  
4          System.out.println("Enrolled: " + lena.enrolledYear);  
5          lena.name = "Lena";  
6          lena.matrNumber = 591753;  
7          lena.enrolledYear = 2012;  
8          System.out.println("Enrolled: " + lena.enrolledYear);  
9      }  
10 }
```

Java

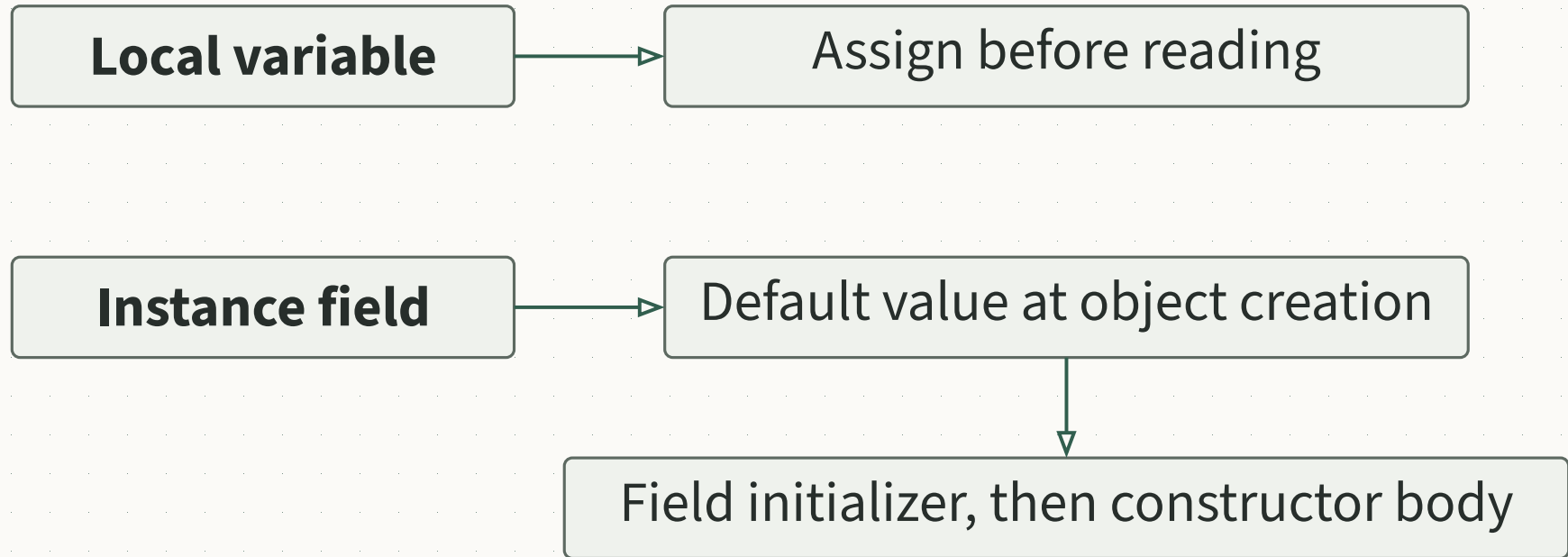
Variables: Scope and Type

Two independent questions



A reference variable can be a local variable or a field.

Initialization



Initialization

Default field values — not defaults for local variables.

Type	Default
byte, short, int, long	0
char	'\u0000'
float, double	0.0
boolean	false
Reference type	null

Initialization

A field initializer replaces the default value.

```
1 class Student {  
2     String name = "Unknown";  
3     int matrNumber;  
4     int enrolledYear = 2019;  
5 }
```

Java

Initialization



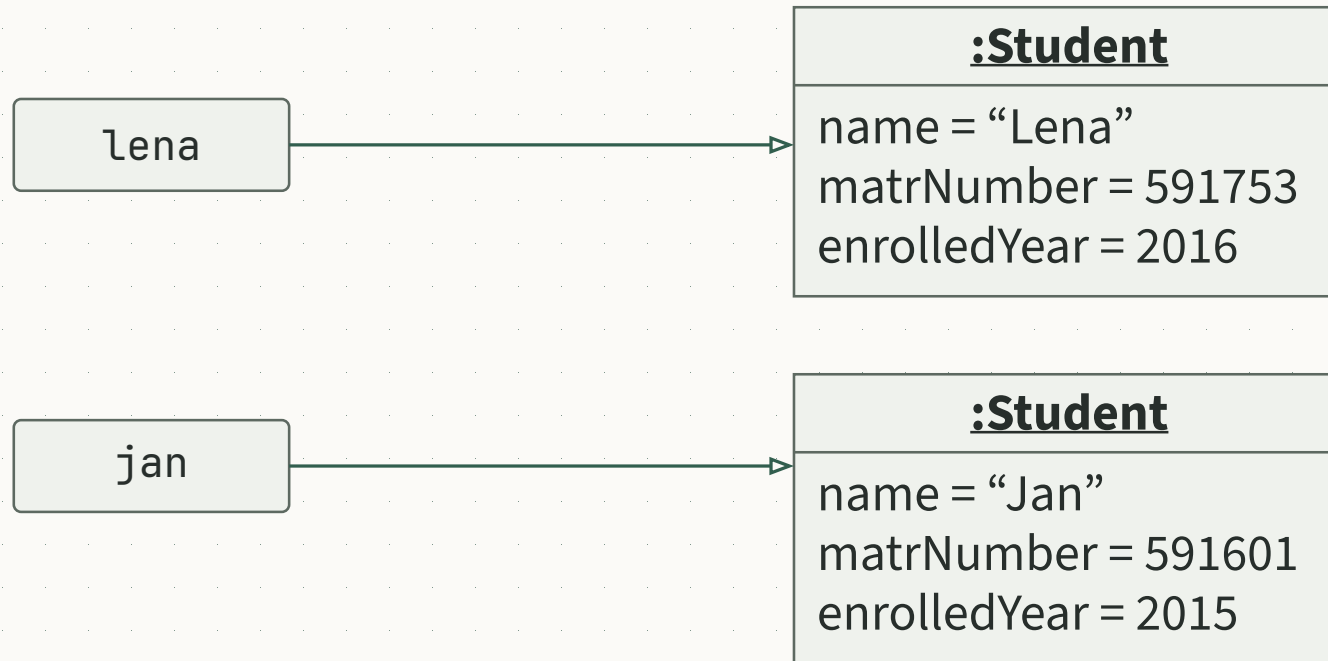
What will this print?

```
1  public class StudentDemo {
2      public static void main(String[] args) {
3          Student lena = new Student();
4          System.out.println("Name: " + lena.name);
5          System.out.println("Number: " + lena.matrNumber);
6          System.out.println("Enrolled: " + lena.enrolledYear);
7          lena.name = "Lena";
8          System.out.println("Name: " + lena.name);
9      }
10 }
```

Java

Assignment of References

Two references, two objects.

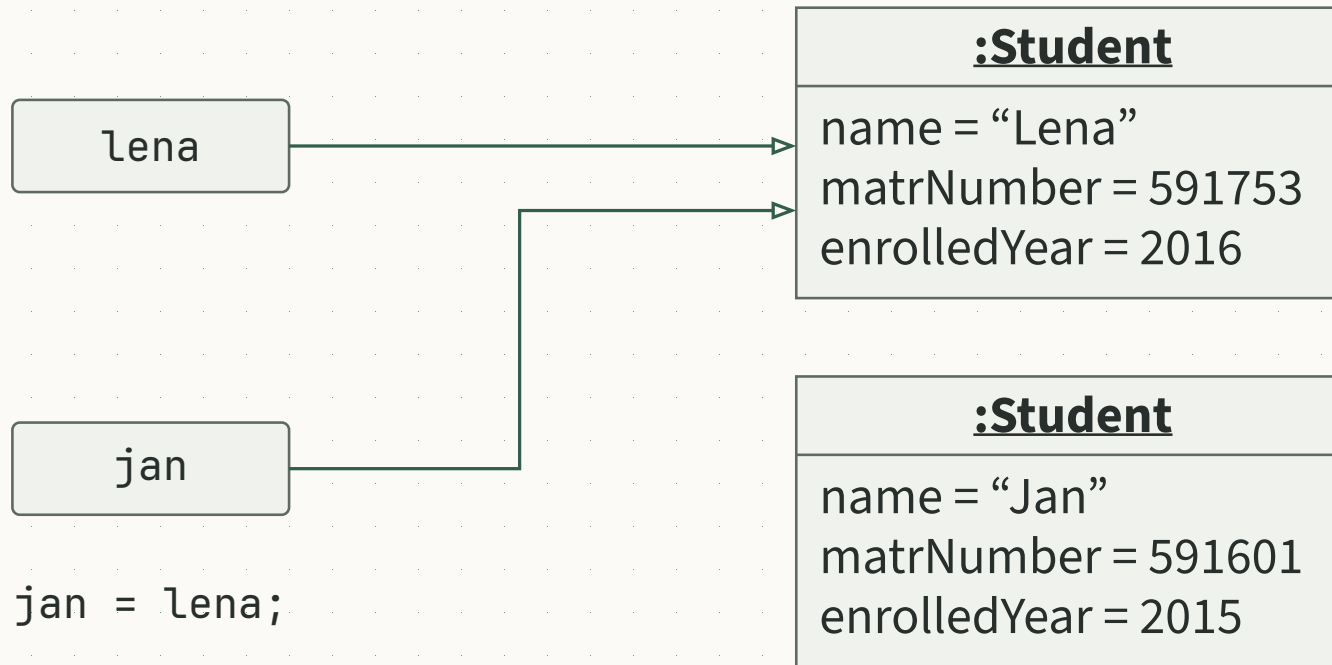


Assignment of References



What changes after `jan = lena;`?

Assignment of References



No remaining reference

Assignment of References



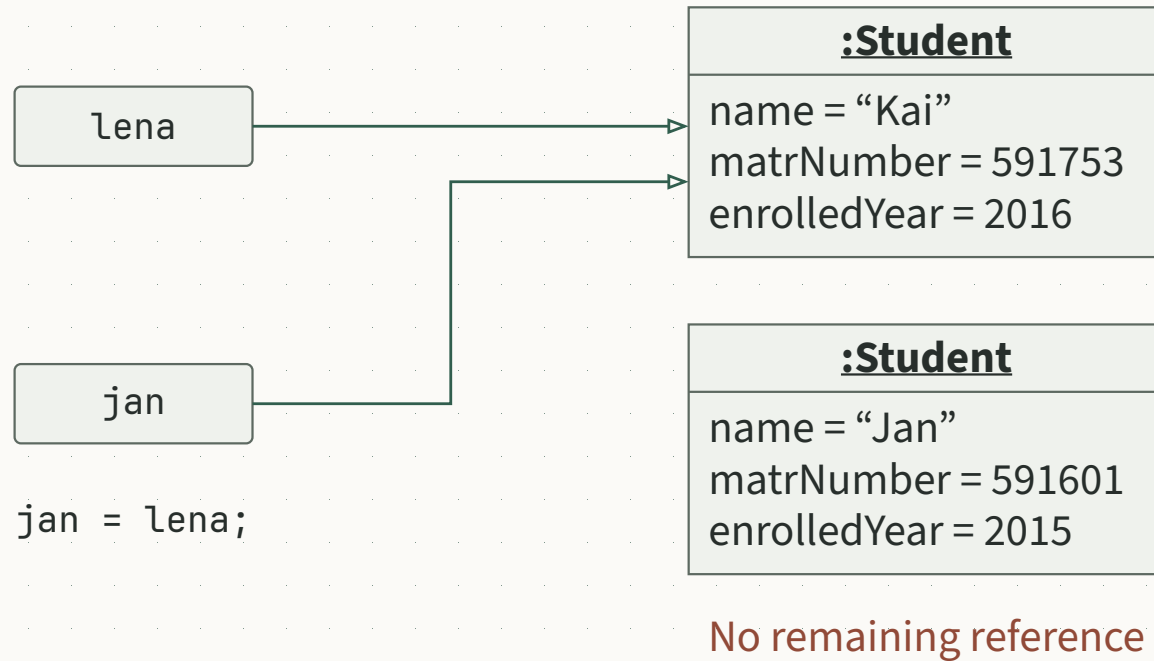
What will this print after `jan = lena;`?

```
1 lena.name = "Birgit";  
2 jan.name = "Kai";  
3 System.out.println(lena.name);  
4 System.out.println(jan.name);
```

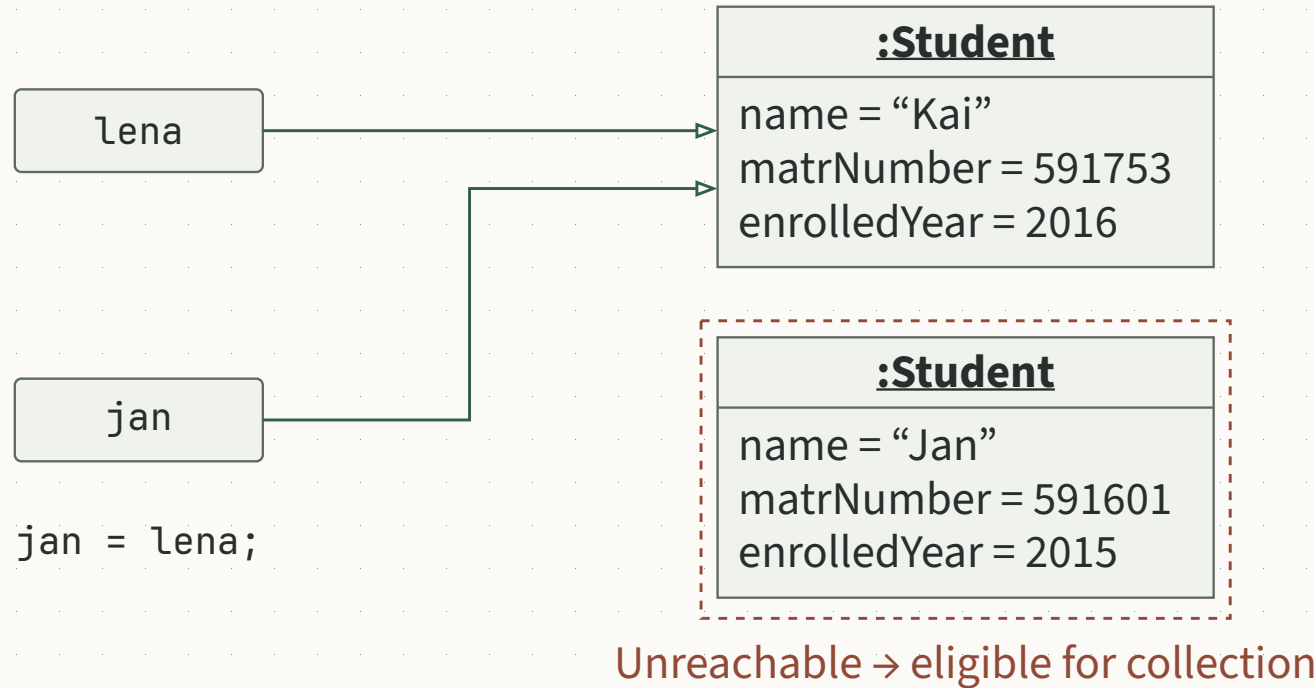
Java

Assignment of References

Both print Kai: the variables refer to the **same object**.



Garbage Collection



Automatic, not immediate. No free or delete.

4. Methods

Methods: Syntax

```
1 ReturnType methodName(Type parameter, ...) {  
2     // Statements  
3     return value;  
4 }
```

Java

ReturnType

↓
Primitive, reference,
or void

methodName

↓
Valid identifier

(Type name, ...)

↓
Parameters, or empty ()

void methods return no value.

Methods: Examples



Task 2

Calculate the average of two floating-point numbers.

Methods: Examples

```
1 public class MathUtils {  
2     double average(double a, double b) {  
3         return (a + b) / 2.0;  
4     }  
5 }
```

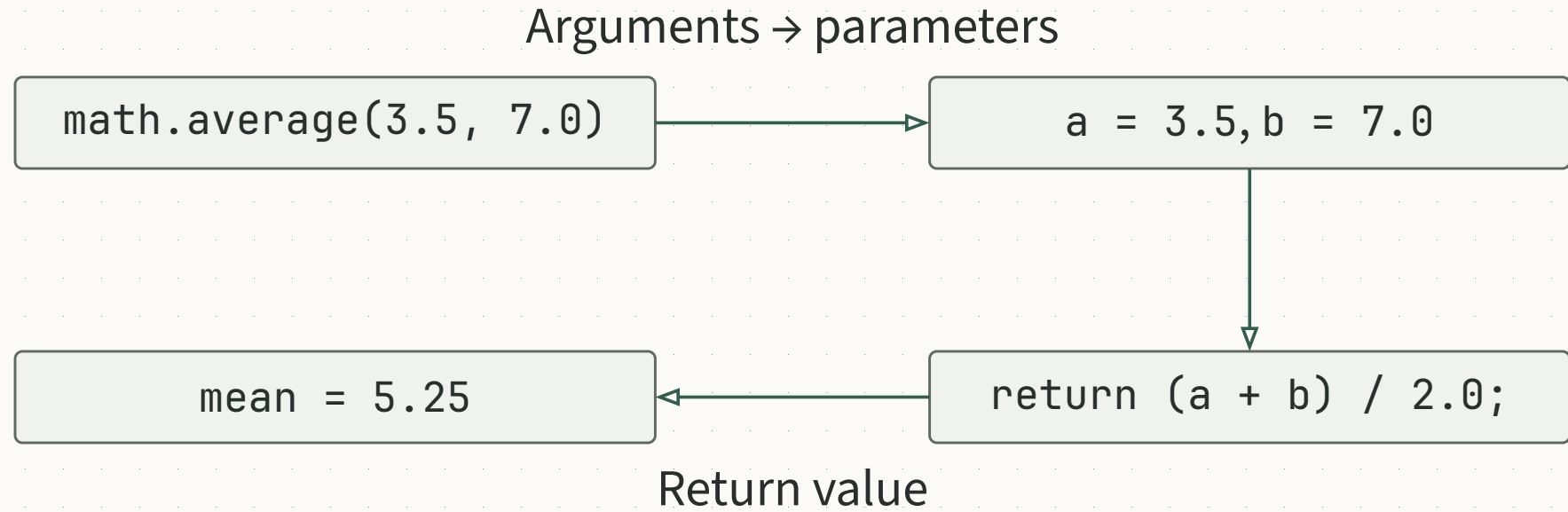
Java

Methods: Examples

```
1 public class MathUtilsDemo {  
2     public static void main(String[] args) {  
3         MathUtils math = new MathUtils();  
4         double a1 = 3.5, a2 = 7;  
5         double mean = math.average(a1, a2);  
6         System.out.println(mean);  
7         System.out.println(math.average(1.5, 3.2));  
8     }  
9 }
```

Java

Methods: Examples



Methods: Examples



Task 3

Calculate the digit sum of a non-negative integer.

Methods: Examples

```
1  public class MathUtils {  
2      int sumOfDigits(int number) {  
3          int sum = 0;  
4          while (number > 0) {  
5              sum += number % 10;  
6              number /= 10;  
7          }  
8          return sum;  
9      }  
10 }
```

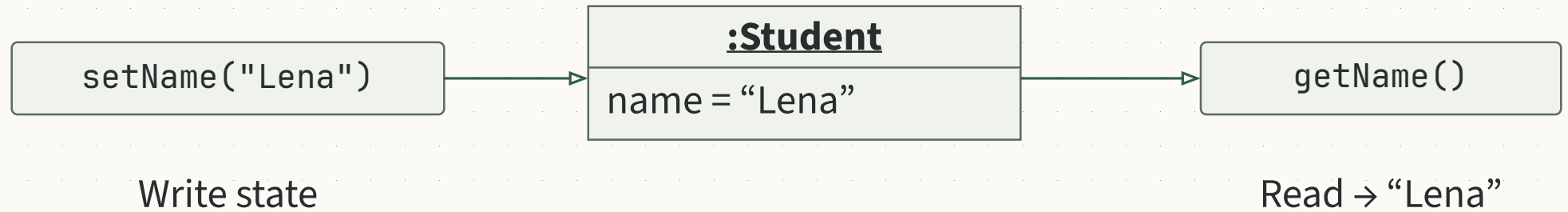
Java

Methods: Examples

```
1 public class MathUtilsDemo {  
2     public static void main(String[] args) {  
3         MathUtils math = new MathUtils();  
4         System.out.println(math.sumOfDigits(0));  
5         System.out.println(math.sumOfDigits(2016));  
6     }  
7 }
```

Java

Getters and Setters



```
1 void setName(String value) {  
2     name = value;  
3 }
```

Java

```
1 String getName() {  
2     return name;  
3 }
```

Java

Methods: Call-by-Value

Java copies argument values into local parameter variables.

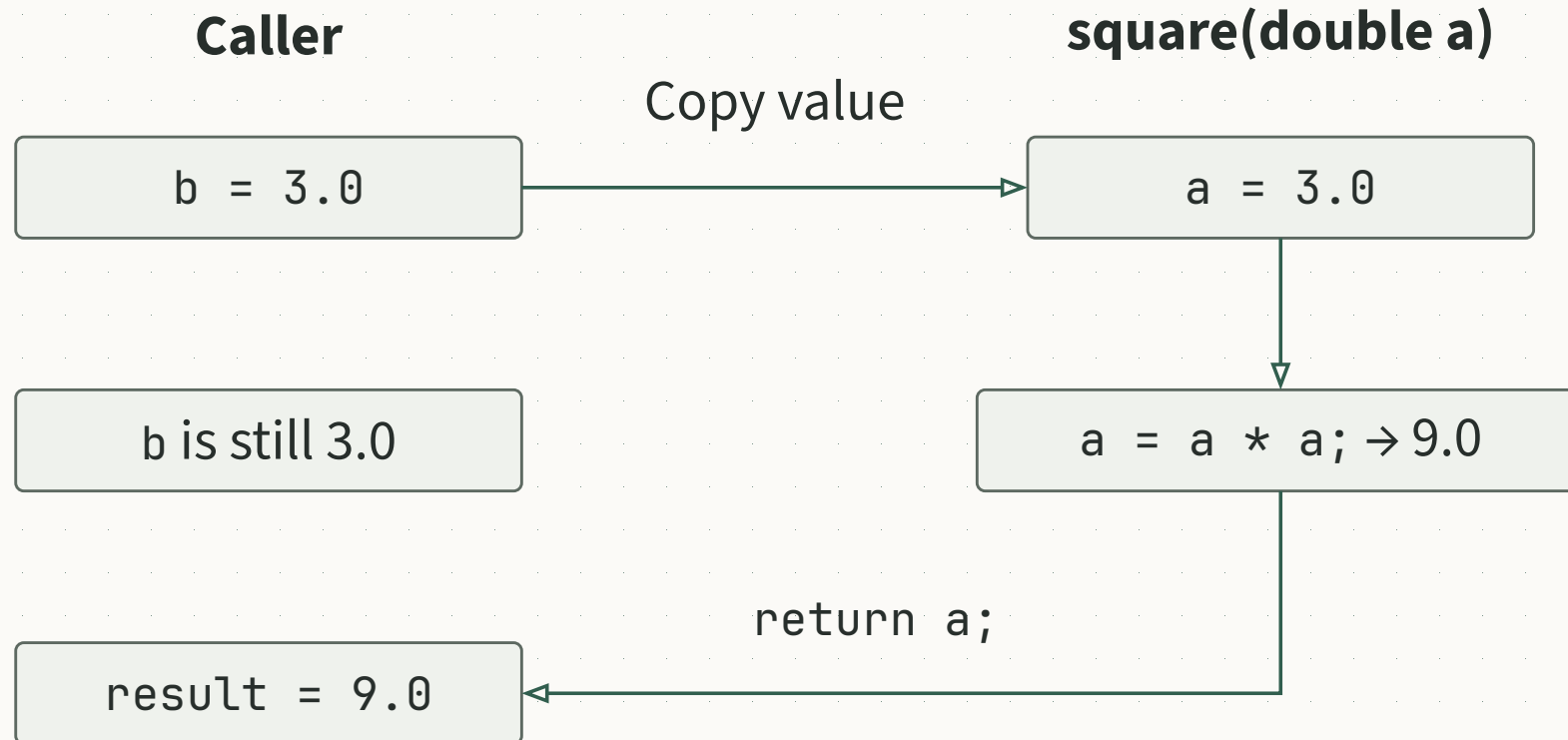
```
1 double square(double a) {  
2     a = a * a;  
3     return a;  
4 }
```

Java

```
1 double b = 3.0;  
2 double result = square(b);
```

Java

Methods: Call-by-Value



Methods: Objects as Parameters

The reference is copied; the object is **not** copied.

```
1 public class CallByValueDemo {  
2     static void setNameBirgit(Student student) {  
3         student.name = "Birgit";  
4         System.out.println(student.name);  
5     }  
6     // main() follows on the next slide  
7 }
```

Java

Methods: Objects as Parameters

```
1 public static void main(String[] args) {  
2     Student lena = new Student();  
3     lena.name = "Lena";  
4     setNameBirgit(lena);  
5     System.out.println(lena.name);  
6 }
```

Java

Both calls to `println` print Birgit.

Methods: Objects as Parameters

Before the call

main()

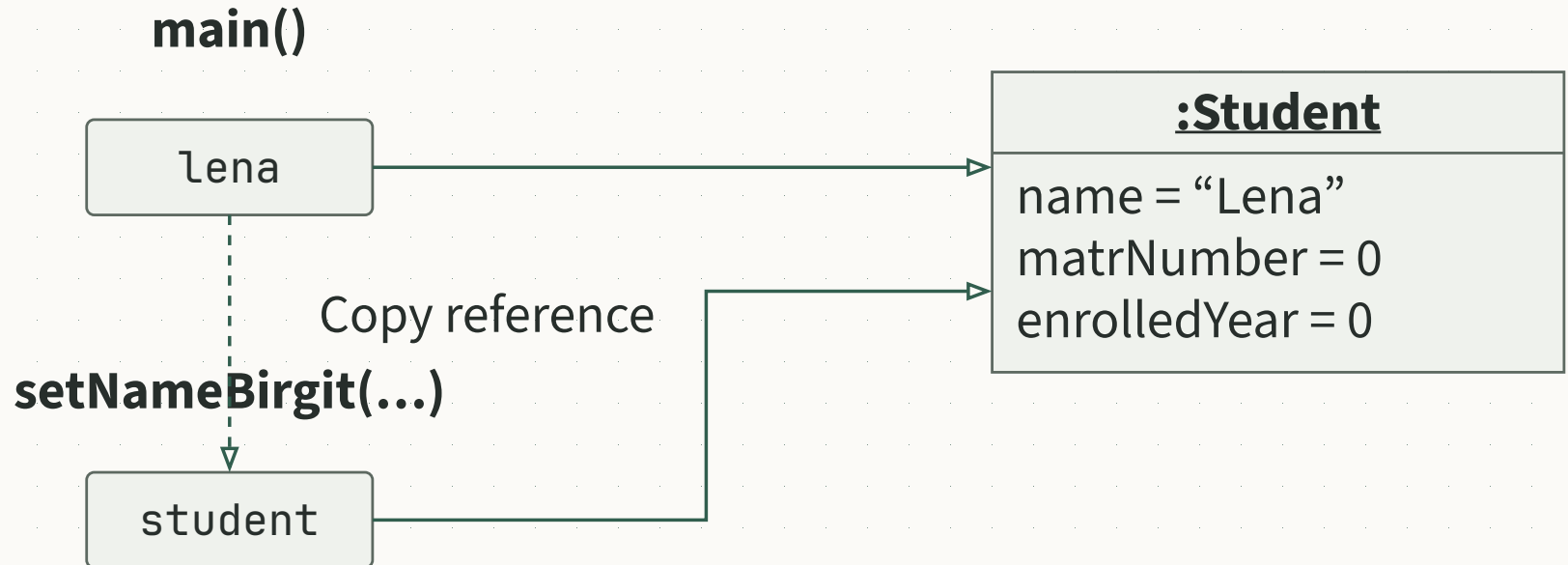
lena

:Student

name = "Lena"
matrNumber = 0
enrolledYear = 0

Methods: Objects as Parameters

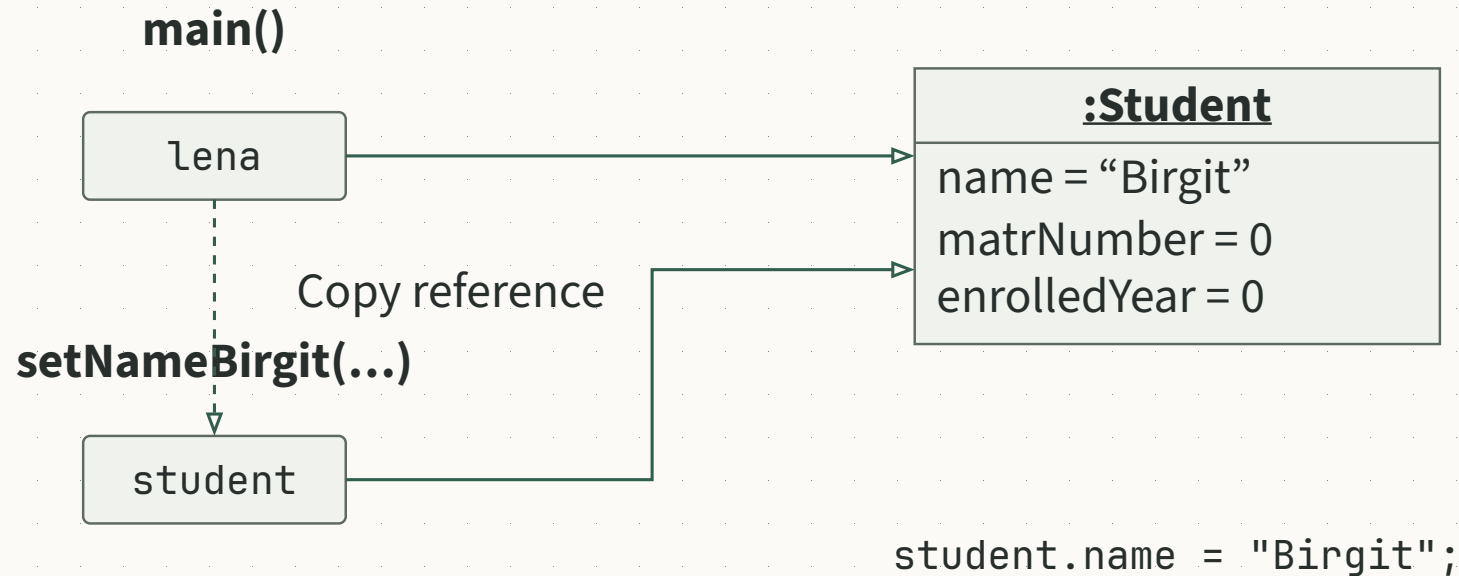
Pass a copy of the reference



Two variables, one object

Methods: Objects as Parameters

Modify the shared object



Reassigning student would not reassign lena.

The `this` Reference



Can the parameter have the same name as the field?

```
1 public class Student {  
2     String name;  
3  
4     void setName(String newName) {  
5         name = newName;  
6     }  
7 }
```

Java

The `this` Reference



Does this change the field?

```
1 void setName(String name) {  
2     name = name;  
3 }
```

Java

The `this` Reference



Does this change the field?

```
1 void setName(String name) {  
2     name = name;  
3 }
```

Java

No. Both names refer to the parameter; it hides the field.

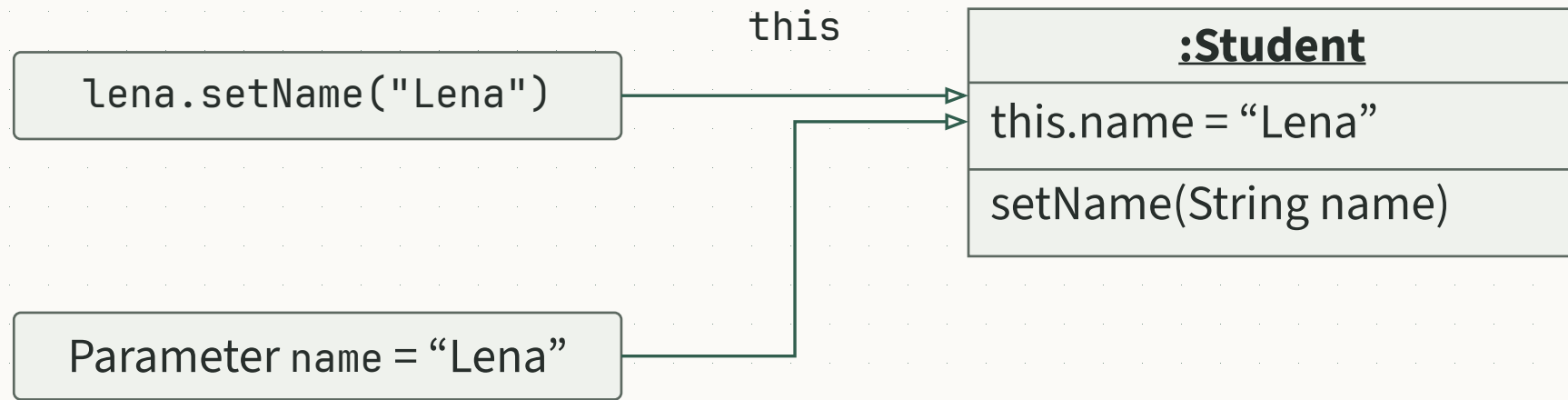
The `this` Reference

`this` refers to the object whose method is running.

```
1 public class Student {  
2     String name;  
3  
4     void setName(String name) {  
5         this.name = name;  
6     }  
7 }
```

Java

The `this` Reference



`this.name = name;` copies the parameter into the field.

Overloading



Task 4

Write a method that returns the maximum of two integers.

```
1  int max(int a, int b) {  
2      if (a > b) {  
3          return a;  
4      } else {  
5          return b;  
6      }  
7  }
```

Java

Overloading



Task 5

Add a method for two double values.

```
1 double max(double a, double b) {  
2     return (a > b) ? a : b;  
3 }
```

Java

Overloading



Task 6

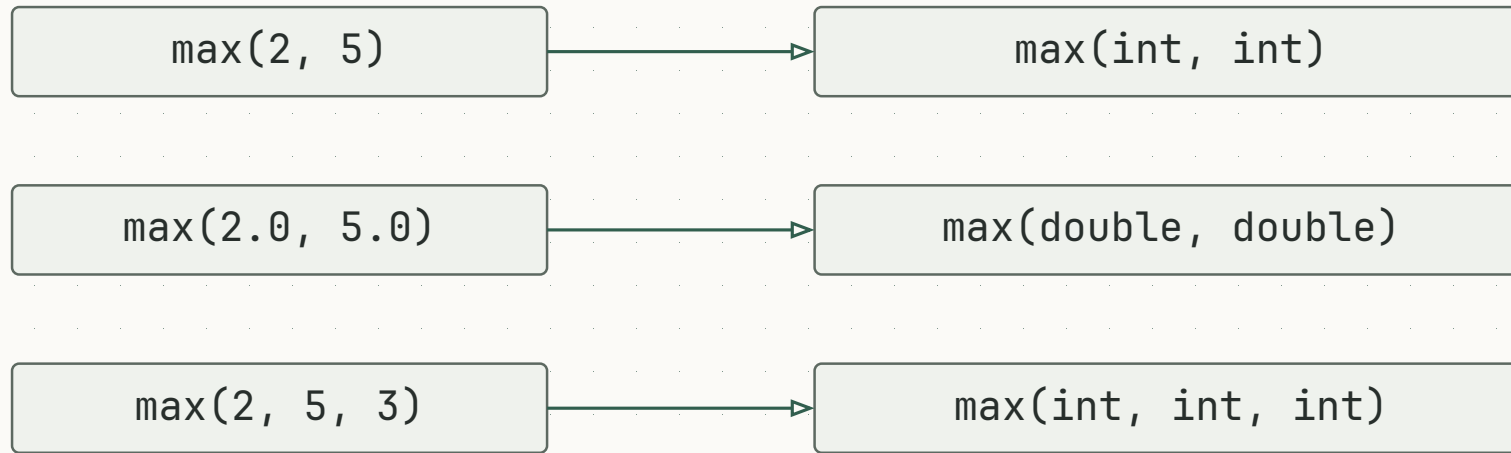
Add a method for three integers.

```
1 int max(int a, int b, int c) {  
2     return max(max(a, b), c);  
3 }
```

Java

Overloading

Compiler selects by argument types and count



Signature = method name + ordered parameter types

Parameter names and return types do **not** distinguish overloads.

Overloading

Same method name does not guarantee the same behavior.

```
1 int max(int a, int b) {  
2     return (a > b) ? a : b;  
3 }  
4  
5 short max(short a, short b) {  
6     System.out.println("Aaaarrrrghhh!");  
7     return 7;  
8 }
```

Java

Methods: Coding Style

Use camelCase (snake_case also accepted here). Use blank lines between methods and logical steps.

```
1 double average(double a, double b) {  
2     return (a + b) / 2.0;  
3 }  
4  
5 int max(int a, int b) {  
6     int result = (a > b) ? a : b;  
7  
8     return result;  
9 }
```

Java

Methods: Coding Style

Use spaces around operators and after commas. Keep indentation consistent.

Readable

```
1 int a = 5;
2 for (int i = 1; i < 10; i++) {
3     a *= i;
4 }
```

Java

Cramped

```
1 int a=5;
2 for(int i=1;i<10;i++){
3     a*=i;
4 }
```

Java

5. Constructors

Constructors

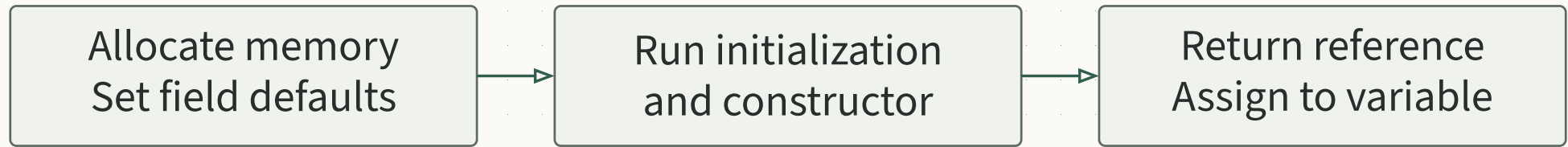
A constructor initializes a new object. It has the class name and **no return type**.

```
1 public class Student {  
2     String name;  
3  
4     Student(String name) {  
5         this.name = name;  
6     }  
7 }
```

Java

Constructors

```
Student lena = new Student("Lena");
```



No-Argument and Default Constructors

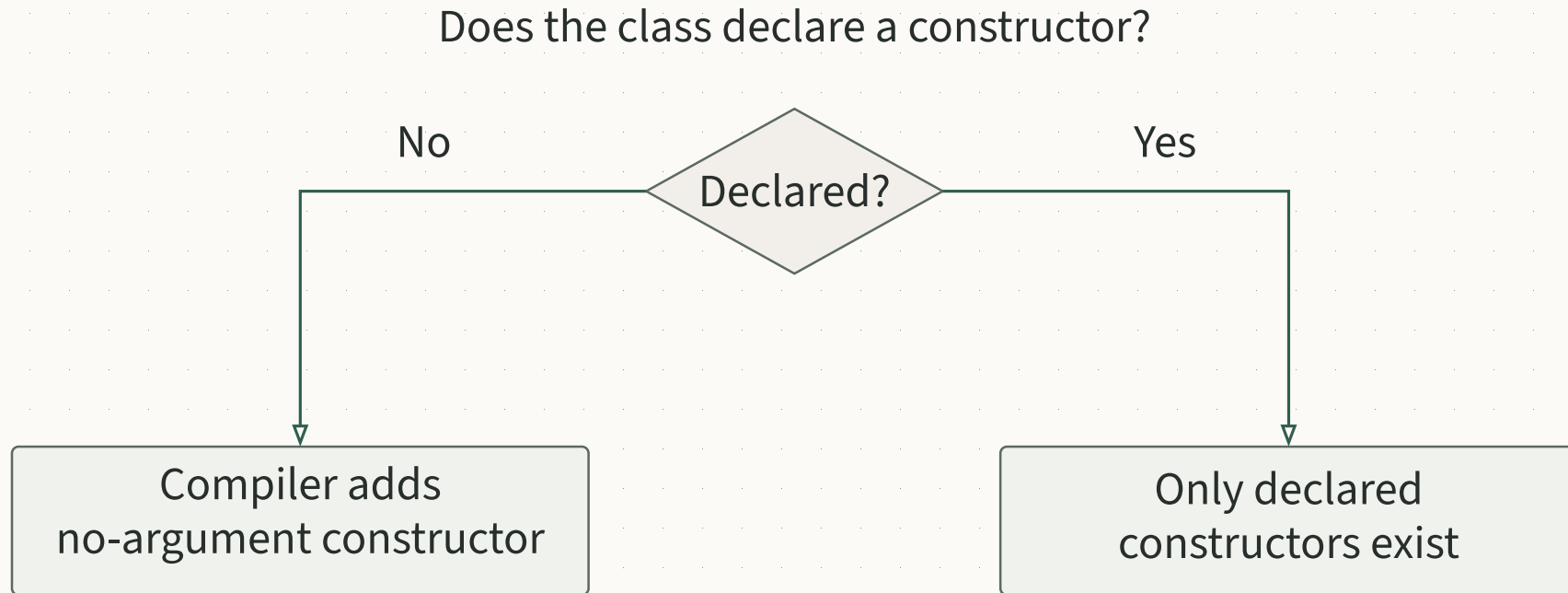
This is an **explicit no-argument constructor**.

```
1 public class Student {  
2     String name;  
3  
4     Student() {  
5     }  
6 }
```

Java

name is already null before the constructor body runs.

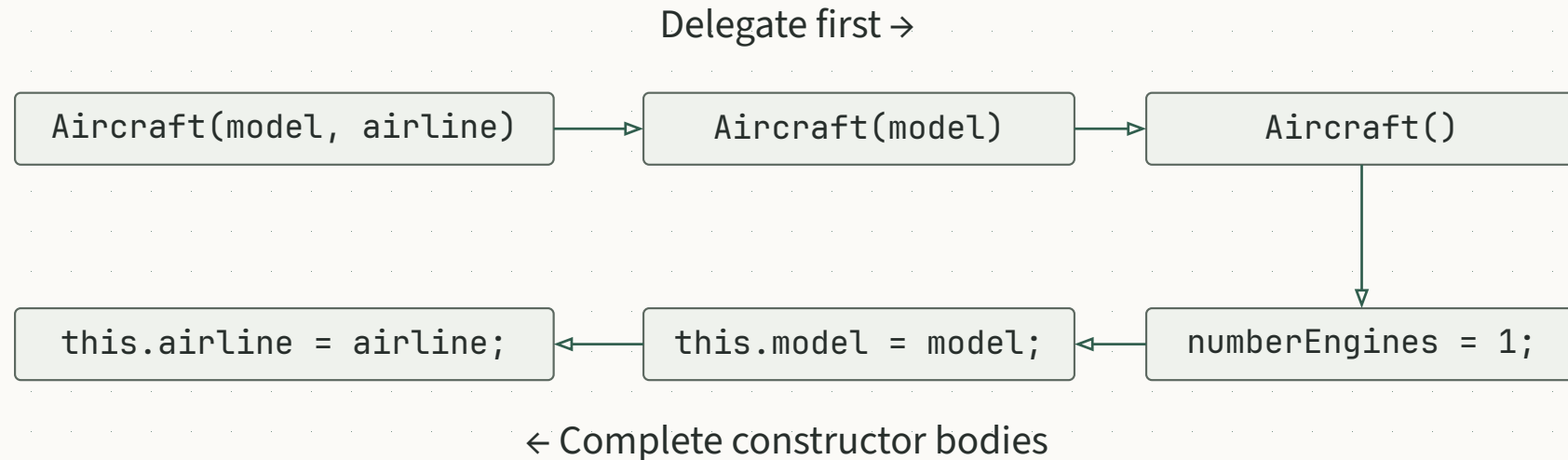
No-Argument and Default Constructors



The compiler-generated no-argument constructor is the **default constructor**.

this VS. this(...)

- `this`: reference to the current object.
- `this(...)`: call another constructor in the same class.
- In these examples, constructor delegation comes first.



this VS. this(...)

```
1 public class Aircraft {  
2     String model, airline;  
3     int numberEngines;  
4  
5     Aircraft() {  
6         numberEngines = 1;  
7     }  
8     // Overloaded constructors follow  
9 }
```

Java

this **VS.** this(...)

Add these constructors inside Aircraft:

```
1 Aircraft(String model) {  
2     this();  
3     this.model = model;  
4 }  
5  
6 Aircraft(String model, String airline) {  
7     this(model);  
8     this.airline = airline;  
9 }
```

Java

Constructors: Examples



Will this compile?

```
1 public class Aircraft {  
2     String model;  
3  
4     public static void main(String[] args) {  
5         Aircraft aircraft = new Aircraft();  
6     }  
7 }
```

Java

Constructors: Examples



Will this compile?

```
1 public class Aircraft {  
2     String model;  
3  
4     public static void main(String[] args) {  
5         Aircraft aircraft = new Aircraft();  
6     }  
7 }
```

Java

Yes. The compiler supplies `Aircraft()`.

Constructors: Examples



Will new Aircraft() compile with this class?

```
1 public class Aircraft {  
2     String model;  
3     Aircraft(String model) {  
4         this.model = model;  
5     }  
6 }
```

Java

Constructors: Examples



Will new Aircraft() compile with this class?

```
1 public class Aircraft {  
2     String model;  
3     Aircraft(String model) {  
4         this.model = model;  
5     }  
6 }
```

Java

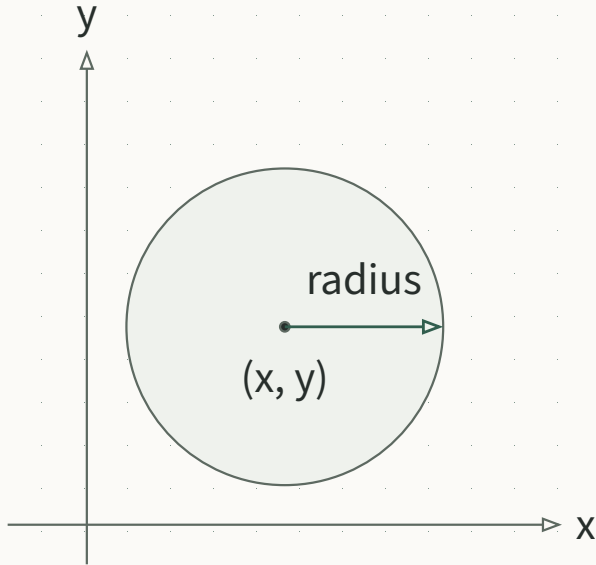
No. Only Aircraft(String) exists; Aircraft() is missing.

Task: Class Circle



Task 7

Model a circle. Implement these four constructors.



Circle
x, y: double radius: double
Circle() → (0, 0, 1) Circle(x, y, radius) Circle(radius) → (0, 0, radius) Circle(circle) → copy fields

Task: Class Circle

```
1  public class Circle {  
2      double x, y, radius;  
3  
4      public Circle(double x, double y, double radius) {  
5          this.x = x;  
6          this.y = y;  
7          this.radius = radius;  
8      }  
9  
10     Circle() {  
11         this(0.0, 0.0, 1.0);  
12     }  
13     // Two more constructors follow  
14 }
```

Java

Task: Class Circle

Add these constructors inside Circle:

```
1 Circle(double radius) {  
2     this(0.0, 0.0, radius);  
3 }  
4  
5 Circle(Circle circle) {  
6     this(circle.x, circle.y, circle.radius);  
7 }
```

Java

The copy constructor creates a **new object** with the same field values.

Task: Class Circle

```
1 public class CircleDemo {  
2     public static void main(String[] args) {  
3         Circle circle1 = new Circle();  
4         Circle circle2 = new Circle(2.5);  
5         Circle circle3 = new Circle(circle2);  
6         Circle circle4 = new Circle(-1.2, 7.1, 3.0);  
7     }  
8 }
```

Java



Info

IntelliJ IDEA: **Code** → **Generate** → **Constructor**.

Task: Class Circle



Task 8

Add getters and setters. Reject negative radii in the setter.



Info

IntelliJ IDEA: **Code** → **Generate** → **Getter and Setter**.

Task: Class Circle

```
1 double getX() {  
2     return x;  
3 }  
4  
5 void setX(double x) {  
6     this.x = x;  
7 }
```

Java

```
1 void setRadius(double radius) {  
2     if (radius ≥ 0.0) {  
3         this.radius = radius;  
4     }  
5 }
```

Java

A negative value leaves the radius unchanged.

Add getY, getRadius, and setY in the same way.

The constructors above do not yet validate the radius.

6. Class Variables & Class Methods

Class Variables



Does PI exist only once in memory?

```
1 public class Circle {  
2     double x, y, radius;  
3     final double PI = 3.141592653589793;  
4  
5     double getArea() {  
6         return PI * radius * radius;  
7     }  
8 }
```

Java

Class Variables

No. `final` prevents reassignment; it does not make a field shared.

:Circle	:Circle	:Circle
x = 2.1 y = -1.4 radius = 2.0 PI = 3.14159...	x = 0.0 y = 0.5 radius = 1.0 PI = 3.14159...	x = -1.0 y = 5.0 radius = 1.5 PI = 3.14159...

Each object has its own field

Class Variables



Does count count all objects created?

```
1 public class Circle {  
2     double x, y, radius;  
3     int count;  
4  
5     Circle() {  
6         radius = 1.0;  
7         count++;  
8     }  
9 }
```

Java

Class Variables

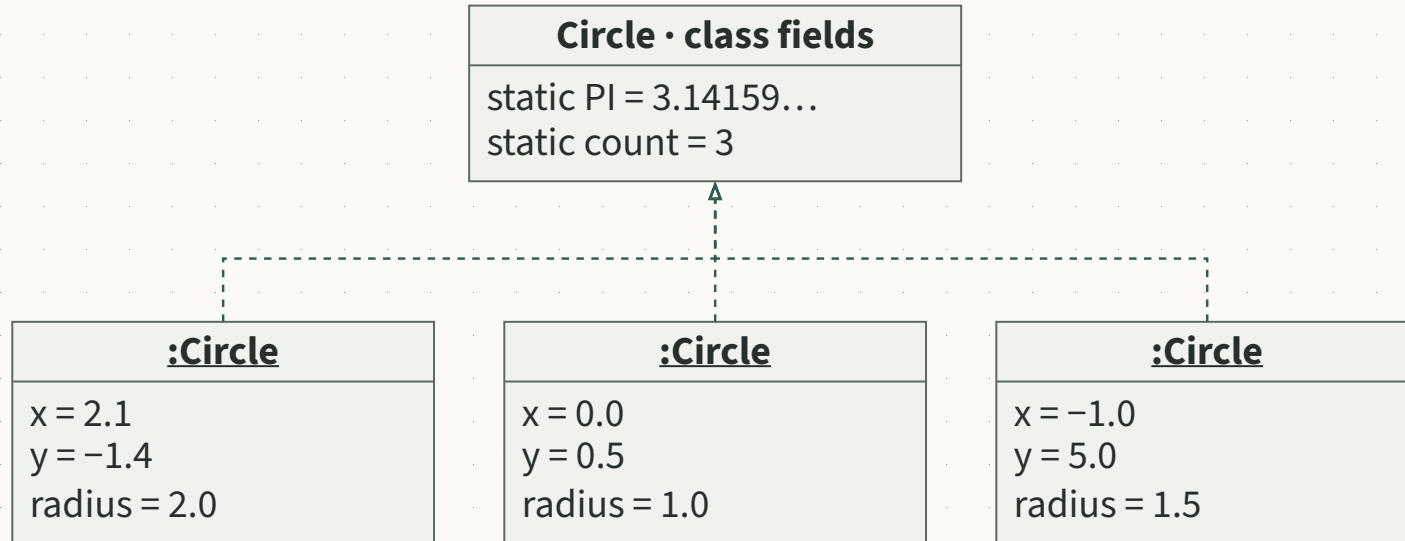
No. Each object increments its own count from 0 to 1.

:Circle	:Circle	:Circle
x = 2.1 y = -1.4 radius = 2.0 count = 1	x = 0.0 y = 0.5 radius = 1.0 count = 1	x = -1.0 y = 5.0 radius = 1.5 count = 1

Each object has its own field

Class Variables

`static` makes a field belong to the **class**, not to each object.



One shared copy per class, even without instances

Class Variables

```
1  public class Circle {  
2      double x, y, radius;  
3      static final double PI = 3.141592653589793;  
4      static int count;  
5  
6      Circle() {  
7          radius = 1.0;  
8          count++;  
9      }  
10  
11     double getArea() {  
12         return PI * radius * radius;  
13     }  
14 }
```

Java

Class Variables

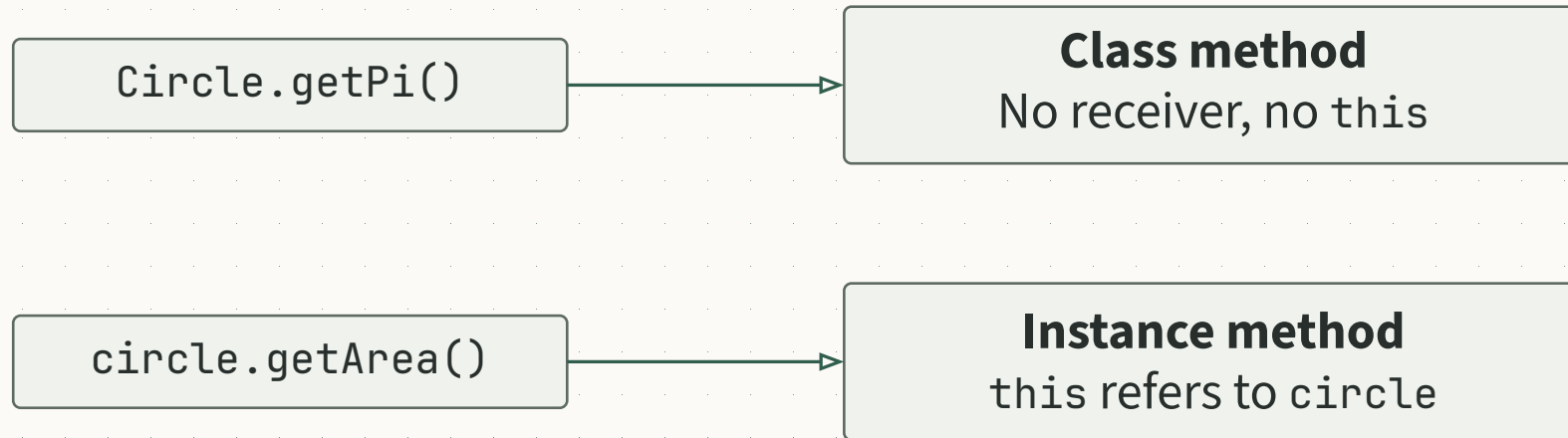
Access a class field with `ClassName.fieldName`.

```
1 public class CircleDemo {  
2     public static void main(String[] args) {  
3         Circle circle1 = new Circle();  
4         Circle circle2 = new Circle();  
5         Circle circle3 = new Circle();  
6  
7         System.out.println("Created: " + Circle.count);  
8     }  
9 }
```

Java

Prints Created: 3. The counter tracks creations, not live objects.

Class Methods



Static methods need an object reference to access instance fields.

Class Methods

```
1 public class Circle {  
2     static double getPi() {  
3         return 3.141592653589793;  
4     }  
5 }
```

Java

```
1 public class CircleDemo {  
2     public static void main(String[] args) {  
3         System.out.println("Pi: " + Circle.getPi());  
4     }  
5 }
```

Java

Both `getPi` and `main` are class methods.

7. License Notice

Attribution

- Shared under CC BY-NC-SA 4.0 and the applicable public license.
- Based on teaching material by Prof. Dr. Marc Hensel.
- Text, diagrams, and layout adapted. Original material supplied privately; no public source link is available.